Mode Opératoire d'Installation — Khazane-DZ

> *Application de Gestion de Stock Intelligente — Agnostique au secteur d'activité*

Table des matières

1. Prérequis
2. Installation – Mode Développement
3. Installation – Mode Production (Docker)
4. Installation – Application Desktop Standalone
5. Vérification de l'installation
6. Dépannage

---

1. Prérequis

Logiciels requis

Logiciel	Version minimale	Téléchargement
Node.js	18.x (LTS)	https://nodejs.org/
npm	9.x (inclus)	Inclus avec Node.js
Docker	24.x	https://www.docker.com/products/docker-desktop
Git	2.x	https://git-scm.com/

Vérification des prérequis

Ouvrir un terminal PowerShell et exécuter :

```
node --version      # doit afficher v18.x ou plus
npm --version       # doit afficher 9.x ou plus
docker --version    # doit afficher 24.x ou plus
git --version       # doit afficher 2.x ou plus
```

Ports utilisés

Service	Port	Remarque
PostgreSQL	5433	Conteneur Docker (mappé depuis 5432)
Redis	6379	Conteneur Docker
Backend API	3002	NestJS
Frontend	5173	Vite dev server (mode développement)

> ⚠️ *Assurez-vous que ces ports ne sont pas déjà utilisés sur votre machine.*

---

2. Installation – Mode Développement

Étape 1 : Cloner le projet

```
git clone <url-du-dépôt> Khazane-DZ
cd Khazane-DZ
```

Étape 2 : Démarrer les conteneurs Docker (PostgreSQL + Redis)

```
npm run docker:up
```

Cela lance :

- **PostgreSQL 16** sur le port `5433` (utilisateur : `khazane`, mot de passe : `khazane_secret`, base : `khazane_db`)
- **Redis 7** sur le port `6379`

Vérifier que les conteneurs tournent :

```
docker ps
```

Vous devez voir `khazane-postgres` et `khazane-redis` avec le statut `Up`.

Étape 3 : Installer les dépendances Backend

```
cd backend
npm install
```

Étape 4 : Configurer les variables d'environnement

Le fichier `backend/.env` est déjà pré-configuré :

```
# Base de données
DATABASE_URL=postgresql://khazane:khazane_secret@localhost:5433/khazane_db

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379

# JWT
JWT_SECRET=change-me-in-production
JWT_EXPIRES_IN=1d

# Application
PORT=3002
NODE_ENV=development
```

> ⚠️ ***En production**, modifiez impérativement* `JWT_SECRET` *avec une valeur aléatoire sécurisée et changez les mots de passe de la base de données.*

Étape 5 : Exécuter les migrations de base de données

```
npx prisma migrate dev
```

Cette commande crée toutes les tables dans PostgreSQL et génère le client Prisma.

Étape 6 : Alimenter la base de données (seed)

```
npx prisma db seed
```

Données créées par le seed :

- **Utilisateur admin** : `admin@khazane.dz` / mot de passe `admin123`
- **Site** : « Entrepôt Principal » (code `ENT-01`), type Warehouse, adresse « Zone industrielle, Alger »
- **Zone** : « Allée 1 » (code `A1`), type AISLE
- **5 emplacements** : `A1-01` à `A1-05` (Étagère 1 à 5), capacité max 100
- **Catégorie** : « Général »
- **Assignation** : Admin assigné au site principal

Étape 7 : Démarrer le Backend

```
npm run start:dev
```

Le serveur API démarre sur `http://localhost:3002`.
La documentation Swagger est accessible sur `http://localhost:3002/api`.

Étape 8 : Installer les dépendances Frontend

Ouvrir un **nouveau terminal** :

```
cd frontend
npm install
```

Étape 9 : Démarrer le Frontend

```
npm run dev
```

Le frontend démarre sur `http://localhost:5173`.

Résumé – Commandes rapides (depuis la racine)

```
npm run docker:up          # 1. Démarrer PostgreSQL + Redis
cd backend && npm install  # 2. Dépendances backend
npx prisma migrate dev     # 3. Migrations
npx prisma db seed         # 4. Données initiales
```

```
npm run start:dev          # 5. Démarrer le backend
# Dans un nouveau terminal :
cd frontend && npm install # 6. Dépendances frontend
npm run dev                # 7. Démarrer le frontend
```

---

3. Installation – Mode Production (Docker)

Build du Frontend

```
cd frontend
npm ci
npm run build
```

Le build est généré dans `frontend/dist/` .

Build du Backend

```
cd backend
npm ci
npx nest build
```

Le build est généré dans `backend/dist/` .

Démarrer le Backend en mode production

```
cd backend
$env:NODE_ENV="production"
$env:DATABASE_URL="postgresql://khazane:khazane_secret@localhost:5433/khazane_db"
$env:JWT_SECRET="votre-secret-production-sécurisé"
$env:PORT="3002"
$env:SERVE_STATIC="true"
node dist/main.js
```

Avec `SERVE_STATIC=true` , le backend sert automatiquement le frontend compilé. L'application complète est alors accessible sur `http://localhost:3002` .

---

4. Installation – Application Desktop Standalone

L'application desktop est un exécutable Windows autonome qui embarque PostgreSQL, le backend et le frontend.

Prérequis supplémentaires

- Connexion Internet (pour le téléchargement initial de PostgreSQL portable ~200 Mo)
- Environ 1 Go d'espace disque libre

Lancer le build automatisé

Depuis la racine du projet :

```
.\scripts\build-desktop.ps1
```

Ce script effectue 6 étapes automatiquement :

Étape	Action
1/6	Téléchargement de PostgreSQL 16 portable
2/6	Build du frontend React
3/6	Build du backend NestJS
4/6	Compilation du script de seed
5/6	Installation des dépendances Electron
6/6	Packaging avec electron-builder (NSIS)

Options du script

```
# Ignorer le téléchargement de PostgreSQL (si déjà présent)
.\scripts\build-desktop.ps1 -SkipPgDownload

# Générer un dossier non packagé (pour tests)
.\scripts\build-desktop.ps1 -DirOnly
```

Résultat

L'installateur Windows (`.exe`) est généré dans le dossier `dist-desktop/` .

Installation sur le poste client

1. Exécuter l'installateur `.exe`
2. Suivre l'assistant d'installation
3. L'application démarre avec un splash screen pendant l'initialisation de la base de données embarquée
4. Aucune installation de PostgreSQL ou Node.js n'est requise sur le poste client

---

5. Vérification de l'installation

Test de connexion

1. Ouvrir le navigateur sur `http://localhost:5173` (dev) ou `http://localhost:3002` (production)
2. Se connecter avec :
   - **Email** : `admin@khazane.dz`
   - **Mot de passe** : `admin123`
3. Vous devez accéder au tableau de bord

Test de l'API

```
# Obtenir un token JWT
$body = '{"email":"admin@khazane.dz","password":"admin123"}'
```

```
$response = Invoke-RestMethod -Uri http://localhost:3002/api/v1/auth/login -Method POST -
Body $body -ContentType "application/json"
$token = $response.access_token

# Tester l'API produits
Invoke-RestMethod -Uri http://localhost:3002/api/v1/products -Headers @{ Authorization =
"Bearer $token" }
```

Outils utiles

```
# Ouvrir Prisma Studio (interface visuelle de la BDD)
npm run db:studio

# Voir les logs des conteneurs Docker
docker logs khazane-postgres
docker logs khazane-redis
```

---

6. Dépannage

Le port 5433 est déjà utilisé

```
# Trouver le processus qui utilise le port
netstat -ano | findstr :5433
# Arrêter le conteneur existant
docker stop khazane-postgres
docker rm khazane-postgres
# Relancer
npm run docker:up
```

Erreur Prisma « Can't reach database server »

Vérifiez que Docker est démarré et que le conteneur PostgreSQL tourne :

```
docker ps | findstr khazane-postgres
```

Si le conteneur n'est pas listé :

```
npm run docker:up
```

Le backend ne démarre pas (port 3002 occupé)

```
netstat -ano | findstr :3002
taskkill /PID <numéro_pid> /F
```

Réinitialiser la base de données

```
cd backend
npx prisma migrate reset
npx prisma db seed
```

> ⚠️ *Cette opération supprime toutes les données existantes.*

Régénérer le client Prisma

Si vous obtenez des erreurs liées à Prisma après un changement de schéma :

```
cd backend
npx prisma generate
```

Document généré pour Khazane-DZ v0.1.0